

School ERP — Operational Guide for Leadership

A business-outcomes guide to what's actually built, grounded in the codebase as it stands today. Every claim below cites the file or schema field it comes from — this is a description of shipped behavior, not a roadmap pitch.

Executive Summary

This system replaces manual fee registers and spreadsheets with a real-time, multi-campus school ERP. It is organized here around five business outcomes rather than a feature list, because leadership doesn't need to know what a "server action" is — they need to know what changes for the front office, for financial trust, and for their own visibility into the school(s) they run.

Outcome	One-line summary
1. Faster Fee Recovery	Collection, receipting, and reminders collapse from minutes to seconds, and defaulters are auto-ranked by risk.
2. Zero-Reload Front Office	The UI updates instantly on every action — no spinners, no full-page reloads.
3. Fraud Prevention & Financial Integrity	Every collection and every record change is permanently, attributably logged.
4. Multi-Campus Governance	One account can run several schools, with data fully isolated between them.
5. Leadership Visibility	Forecasts, trends, and exports are available on demand, without asking accounts for a report.

A companion slide deck, [leadership-presentation.html](#), covers the same material for a live briefing — open it in any browser (no install, no internet connection required).

Outcome 1 — Faster Fee Recovery

The end-to-end flow

Fee collection is a single Server Action, `collectFee` in `app/(app)/fees/actions.ts`. Walking through what it actually does, in order:

1. **Validates the payment** — amount must be positive and finite, must not exceed the remaining balance (with a 0.01 floating-point tolerance for repeated partial payments), and the payment mode must be one of `CASH`, `UPI`, `BANK_TRANSFER`, or `CHEQUE` (`prisma/schema.prisma`, `PaymentMode` enum).
2. **Requires a reference for UPI** — `REFERENCE_REQUIRED_MODES` forces a transaction ID before a UPI payment can be recorded at all.
3. **Runs atomically** — the whole operation (re-reading the due, counting existing receipts for numbering, creating the receipt, updating the due's `amountPaid` / `isPaid`) happens inside one `prisma.$transaction`, so a receipt is never created without the due being updated, or vice versa.
4. **Auto-numbers the receipt**, namespaced per school: `RCPT-{school.code}-{year}-{sequence}`, e.g. `RCPT-GWS-2026-0042`.
5. **Supports partial payment** — if the amount doesn't cover the full due, the due stays open with an updated `amountPaid`, and the UI clearly labels it a partial payment with the remaining balance.
6. **Writes an audit log entry** (`logAudit`, action type `FEE_COLLECTED`) recording the receipt number, amount, full/partial status, fee title, student name, and payment mode.
7. **Fires a WhatsApp receipt confirmation** — see the note on this below; it's non-blocking by design (`.catch(() => {})`) so a WhatsApp failure can never prevent a fee from being recorded.
8. **Revalidates** `/fees`, `/dashboard`, and `/students` so every dependent view reflects the new numbers immediately.

Defaulter risk scoring

`lib/analytics/defaulters.ts` computes a deterministic risk score for every student with a pending due — no ML, no black box, fully explainable:

Signal	Weight	How it's capped
Max days overdue on any pending due	50%	Capped at 90 days, so one very old due can't single-handedly dominate the score
Number of simultaneous pending dues	25%	Capped at 3
Historical ratio of late vs. on-time past receipts	25%	Uncapped ratio (0–1)

Scores ≥ 55 are labeled **High risk** (red badge), ≥ 25 **Medium risk** (amber badge), below that unbadged — intentionally, so the badge draws attention only where it's warranted

(`app/(app)/fees/pending-dues-table.tsx`). This runs on every load of the Pending Dues table and requires no manual review process.

One-click reminders

`components/fees/whatsapp-reminder-button.tsx` builds a pre-written reminder message (student name, fee, amount, due date, school name) and opens `wa.me/<parent phone>?text=<message>` directly in a new tab. This is real, client-side, and requires no backend integration — it works today with no configuration.

WhatsApp receipt confirmation — built, not yet live

Separately, `lib/whatsapp.ts` builds the **exact** payload shape the Meta WhatsApp Business Cloud API expects for a receipt-confirmation template message, and logs it to the server console on every successful collection instead of sending it. The code comment is explicit about this: swap the `console.log` for a `fetch` to Meta's Graph API once `WHATSAPP_ACCESS_TOKEN` and `WHATSAPP_PHONE_NUMBER_ID` are available — no other call site needs to change. **This is a configuration step away from live, not new development.**

Why "3× faster" is a workflow estimate, not a benchmark

The old process for one family typically runs: manual ledger lookup (2–3 min) → hand-written/typed receipt with manual balance math (2–3 min) → manually drafted reminder message (1–2 min) → no systematic prioritization of who to chase. The new process: type-ahead student search auto-loads live dues (seconds) → one click collects and auto-numbers the receipt → one click sends a pre-written reminder → risk badges auto-rank who matters most. Collapsing 5+ minutes of manual work per family into under 20 seconds, while also fixing the "who to chase first" guesswork, is where the 3× estimate comes from. It is a directional, workflow-derived figure — validate it against your own time-motion data after go-live, not as an audited claim.

Outcome 2 — Zero-Reload Front Office

Optimistic UI on fee collection

`app/(app)/fees/fees-client.tsx` uses React 19's `useOptimistic` (`applyOptimisticPayment`) to update the Pending Dues table the instant a cashier clicks "Collect" — before the server has confirmed anything. If the due is now fully paid, it's removed from

the list immediately; if partial, its remaining amount updates immediately. The server call happens in the background inside a `startTransition`, and the UI reconciles automatically.

Animated, count-up numbers

`components/ui/animated-counter.tsx` uses Framer Motion (`animate`, `useMotionValue`, `useInView`) to animate every dashboard stat — Total Students, Monthly Fee Collected, Pending Dues, Today's Attendance Rate — counting up from zero the first time it scrolls into view, rather than the number simply appearing. This is used throughout `components/stat-card.tsx` on the dashboard.

Command palette

`components/command-palette.tsx` binds `Ctrl/Cmd+K` to a searchable command list (navigation + quick actions like "Collect Fee" and "Mark Attendance"), filtered to only what the signed-in user's role can access (`canAccess` from `lib/rbac.ts`). Selecting an action does a client-side route push — no reload.

Performance-conscious code splitting

The fee-collection sheet (`CollectFeeSheet`) is loaded via `next/dynamic` only when a user actually opens it — it's not in the initial page bundle (`app/(app)/fees/fees-client.tsx`). Similarly, the PDF export libraries (`html2canvas`, `jspdf`) are dynamically imported only when a user clicks "Export PDF" on a receipt (`components/fees/receipt-modal.tsx`), keeping the app's baseline JS payload lean.

Instant, flexible receipts

`components/fees/receipt-modal.tsx` renders a receipt with a live toggle between full A4 and an 80mm thermal-counter layout, prints via the browser's native print dialog, and exports to PDF entirely client-side (`html2canvas` rasterizes the receipt DOM, `jsPDF` packages it) — no server round-trip for either. It also fetches and displays the family's total outstanding balance across all other dues (`getStudentOutstandingBalance`), so a parent sees the full picture, not just this one payment.

Outcome 3 — Fraud Prevention & Financial Integrity

Two independent trails

Financial trust here comes from two separate, mutually-reinforcing records:

1. The **FeeReceipt** row itself (`prisma/schema.prisma`) is immutable once created and always stores `collectedBy` , `paymentMode` , and `transactionId` — and, as noted above, UPI payments cannot be recorded without a transaction reference at all.
2. A structured **AuditLog** entry, written by `logAudit()` in `lib/audit.ts` on every mutating action across the app — fee collection, student create/update/delete, attendance marking, settings updates, and fee-category create/update/delete (the full `AuditActionType` enum has nine values). Each entry captures the school, the acting user, the action type, the target entity and ID, a human-readable summary string, the timestamp, and — where available — the acting user's IP address (read from `X-forwarded-for`).

The Audit Logs screen

`app/(app)/audit-logs/` renders the last 100 entries for the active school, with:

- Free-text search across user name and detail text, plus filters by action type and target entity (`audit-log-table.tsx`)
- Auto-refresh every 20 seconds (`router.refresh()` on an interval) so the view stays current without a manual reload
- Sortable columns for user and time, with both a relative ("3 hours ago") and absolute, hoverable timestamp
- Gated entirely behind the `DIRECTOR` role in `lib/route-access.ts` — Admins, Teachers, and Parents cannot see it, by design

Access control is enforced, not just hidden

`lib/rbac.ts` defines what each of the four roles (`DIRECTOR` , `ADMIN` , `TEACHER` , `PARENT`) can access, and every restricted Server Component page calls `requireRouteAccess(key)` (`lib/route-access.ts`) at the top of the page — this redirects to `/login` if there's no valid session, or to `/dashboard` if the signed-in user's role isn't permitted. This is a server-side check on every page load, not a client-side menu that merely hides a link.

Session and tenant integrity

`lib/school-context.ts` never trusts the session cookie's `schoolId` at face value: `getActiveSchoolContext()` re-verifies it against a real `UserSchool` membership row on every call. A stale or tampered cookie pointing at a school the user doesn't belong to resolves to `null`, not someone else's data. The same re-verification happens in `switchActiveSchool()` (`lib/school-actions.ts`) before a school switch is honored.

Outcome 4 — Multi-Campus Governance

Tenant architecture

`School` is the tenant root in `prisma/schema.prisma` — every business record (`Class`, `Student`, `FeeStructure`, `FeeDue`, `FeeReceipt`, `Attendance`, `AuditLog`, `FeeCategoryRule`) carries a `schoolId` foreign key, and every query in the app filters by the active school's ID. A `UserSchool` join table lets one person hold **different roles at different schools** (e.g. Director at one campus, Teacher at another) — role lives on the membership, not on the user.

Isolation guarantees

- Every tenant-scoped Prisma query (fees, students, receipts, audit logs) filters explicitly by `schoolId` — there is no code path that queries across schools by accident.
- The print-receipt page (`app/(print)/receipts/[receiptId]/page.tsx`) looks up the receipt with `findFirst({ where: { id, schoolId } })` — a receipt ID belonging to a different school simply doesn't resolve, returning a 404 rather than leaking that the record exists elsewhere.
- Receipt numbers are namespaced by the school's unique `code` (e.g. `GWS`, `RIS`), so two campuses issuing receipts on the same day never collide on the database's global uniqueness constraint on `receiptNo`.

The school switcher

`components/school-switcher.tsx` only renders as an actual dropdown when the signed-in user has more than one school membership — with exactly one, it shows the school name plainly with no switching affordance to avoid clutter. Switching re-verifies the target membership server-side and creates a fresh session before redirecting.

Demonstrated, not just designed

`prisma/seed.ts` seeds two real schools — Greenwood School (`GWS` , Mumbai) and Riverside International School (`RIS` , Bengaluru) — with a shared Director account (`director@greenwoodsschool.edu`) holding a `DIRECTOR` membership at both, specifically so the switcher can be demonstrated with real, isolated data rather than a single-tenant stub.

Outcome 5 — Leadership Visibility

`app/(app)/dashboard/analytics.ts` and `app/(app)/dashboard/page.tsx` compute, on every dashboard load:

- **A run-rate forecast for the current month** — collections so far scaled by day-of-month progress to a projected full-month total, then compared against last month's actual full-month total for a fair, apples-to-apples growth percentage (`getDashboardForecast`).
- **A trailing 6-month view of collected vs. pending amounts**, rendered as a bar chart (`getMonthlyFinancials` , `components/charts/collections-bar-chart.tsx`), so leadership sees the trend shape, not just a snapshot.
- **A fee-category breakdown** (proxied today by fee-structure title, since there's no separate category field yet — see Limitations) rendered as a donut chart, folding long tails into "Other" beyond the top 6 categories (`getFeeCategoryBreakdown` , `components/charts/fee-category-donut.tsx`).
- **Today's attendance rate**, alongside total students and pending dues, as headline stat cards.

On-demand CSV exports are available with no server action round-trip —

`app/(app)/fees/receipts/export/route.ts` and `app/(app)/fees/structures/export/route.ts` are plain streamed HTTP routes, scoped to the active school, producing RFC 4180-compliant CSV (`lib/csv.ts`) for receipts and the full per-student fee ledger respectively.

Data Model Reference

Grounded directly in `prisma/schema.prisma` :

Model	Purpose
<code>School</code>	Tenant root. Unique <code>code</code> slug namespaces receipt numbering; holds branding (name, logo, address, contact), <code>currency</code> , and <code>currentAcademicYear</code> .
<code>User</code> / <code>UserSchool</code>	A person and their (possibly multiple) school memberships, each with its own <code>Role</code> .
<code>Class</code>	A section within a school (e.g. "9-A").
<code>Student</code>	Belongs to a school and a class; carries <code>admissionNo</code> (globally unique), parent contact, and an <code>isDiscounted</code> flag.
<code>FeeStructure</code>	A fee assigned to a class with an <code>amount</code> and <code>dueDate</code> . <i>(Currently seeded directly — see Limitations.)</i>
<code>FeeDue</code>	One student's obligation against a <code>FeeStructure</code> ; tracks <code>dueAmount</code> , <code>amountPaid</code> , and <code>isPaid</code> .
<code>FeeReceipt</code>	An immutable collection record: amount, payment mode, <code>collectedBy</code> , optional <code>transactionId</code> , auto-generated <code>receiptNo</code> .
<code>Attendance</code>	One student's status (<code>PRESENT</code> / <code>ABSENT</code> / <code>LATE</code> / <code>LEAVE</code>) for one date, unique per student per day.
<code>AuditLog</code>	Structured trail of every mutating action, with actor, action type, target, details, and IP.
<code>FeeCategoryRule</code>	A configurable catalog of fee types (name, amount, frequency, late-fee %) managed in Settings. <i>(Not yet wired to <code>FeeStructure</code> — see Limitations.)</i>

Roles & access matrix (`lib/rbac.ts`)

Role	Dashboard	Students	Classes	Attendance	Fees	Audit Logs	Settings
Director	✓	✓	✓	✓	✓	✓	✓
Admin	✓	✓	—	—	✓	—	—
Teacher	✓	—	✓	✓	—	—	—
Parent	✓	—	—	—	—	—	—

Current Limitations & Production Readiness Roadmap

In the spirit of "benefits over features," this section is deliberately candid — these are the gaps between what's demonstrated today and a full production rollout.

1. **Sign-in is a demo role-picker, not real authentication.** `app/login/actions.ts` (`loginAs(role)`) simply signs in as the first user holding that role — there is no password, magic link, or SSO yet. This must be replaced with real credential- or SSO-based authentication before handling real student data.
2. **WhatsApp receipt confirmations are mocked.** As described above, the payload is production-shaped and logged, but not actually sent — this needs real Meta WhatsApp Business API credentials wired in (`lib/whatsapp.ts`).
3. **FeeCategoryRule (Settings) isn't yet connected to FeeStructure (what actually drives dues).** Today, fee categories can be configured in Settings, but the `FeeStructure` records that generate real student dues are only created via the database seed script — there is no in-app UI yet to assign a fee structure to a class. This is the next logical feature to close the loop between "configure fee types" and "bill students."
4. **Currency display is hardcoded to INR.** `School.currency` exists in the schema, but `lib/currency.ts` 's `formatINR` doesn't yet branch on it — every amount in the UI renders in Indian Rupees regardless of the school's configured currency.
5. **Receipt numbering under very high concurrency.** Numbering is computed as a per-school count inside the collection transaction — safe for the normal single- or few-cashier flow, but schools running many simultaneous collection counters at exactly the same moment should consider migrating to a database sequence for the receipt number to eliminate any theoretical race.
6. **No automated test suite is present in the repository today.** Correctness currently rests on manual verification and the type system; adding test coverage for the collection transaction and defaulter scoring logic is recommended before scaling usage.

None of these block using the system for a guided pilot or demonstration — they define the punch list for a production go-live.

Deployment & Infrastructure

- **Framework:** Next.js 16 (App Router), React 19, Server Actions for all writes — no separate REST/GraphQL API layer to keep in sync.
- **Database:** PostgreSQL via Prisma 7 with the `@prisma/adapters-pg` driver adapter, configured for Supabase (`lib/prisma.ts` , `prisma.config.ts`). The pooled connection (Supavisor, port 6543) is used at runtime; the direct connection (port 5432) is used for migrations, since pooled connections don't support the prepared statements migrations need (`.env.example`).
- **Styling & UI:** Tailwind CSS v4, shadcn-based component primitives (`@base-ui/react`), Framer Motion for animation, Recharts for charts, TanStack Table for data grids.

- **Client-side utilities:** `html2canvas` + `jsPDF` for receipt PDF export, `cmdk` for the command palette, `sonner` for toast notifications.
-

This guide reflects the codebase as of the "Phase 3" milestone (multi-tenancy, school switcher, defaulter risk engine, and analytics). Regenerate or update it as the system evolves — particularly once the items in the Limitations section are closed out.